

mainloop outline/vars.inc

```
1 ; Variable Declarations
2 ; Houses all register definitions, SRAM allocations, and constant equates for the pinball machine.
3
4 ; EEPROM addresses to store the high score in non-volatile memory
5 .equ HIGH_SCORE_ADDR      = 0x00
6
7 ; game state simple binary flags to track if we are actively playing or done
8 .equ PLAYING              = 0x00
9 .equ GAME_OVER            = 0x01
10
11 ; game control sensors w/ specific hex codes for buttons that control the flow of the game
12 .equ SENSOR_SELECT_PLAYERS = 0x01
13 .equ SENSOR_START_GAME     = 0x02
14 .equ SENSOR_RESET          = 0x03
15 .equ SENSOR_TILT           = 0x04
16 .equ SENSOR_BALL_OUT       = 0x05
17
18 ; actuator sensors defining a range for the 8 actuator buttons
19 .equ SENSOR_ACTUATOR_MIN   = 0x10
20 .equ SENSOR_ACTUATOR_MAX   = 0x17
21
22 ; scoring sensors defining a range for general playfield targets/bumpers
23 .equ SENSOR_SCORE_MIN      = 0x20
24 .equ SENSOR_SCORE_MAX      = 0x3F
25
26 ; audio freqs hz constants for the PlayNote function so it's easy to tune the sounds later
27 .equ GAME_START_TONE       = 880
28 .equ ACTUATOR_TONE         = 440
29 .equ SCORE_TONE            = 523
30 .equ TILT_TONE             = 150
31 .equ BALL_OUT_TONE         = 200
32 .equ GAME_OVER_TONE        = 100
33
34 ; data segment
35
36 .dseg
37 .org SRAM_START
38 ; 8 bytes total 4 players and 2 bytes each for a 16-bit score
39 Player_Scores: .byte 8
40 ; tracks whose turn it is 1->4
41 CurrentPlayer: .byte 1
42 ; tracks which ball the current player is on 1->5
43 CurrentBall:   .byte 1
44 ; tracks how many people are playing this match 1->4
45 ActivePlayers: .byte 1
46 ; tracks if the game state is PLAYING or GAME_OVER
47 GameState:     .byte 1
48 ; 2 bytes to hold the all-time high score loaded from EEPROM
49 HighScore_Variable: .byte 2
```